Contents

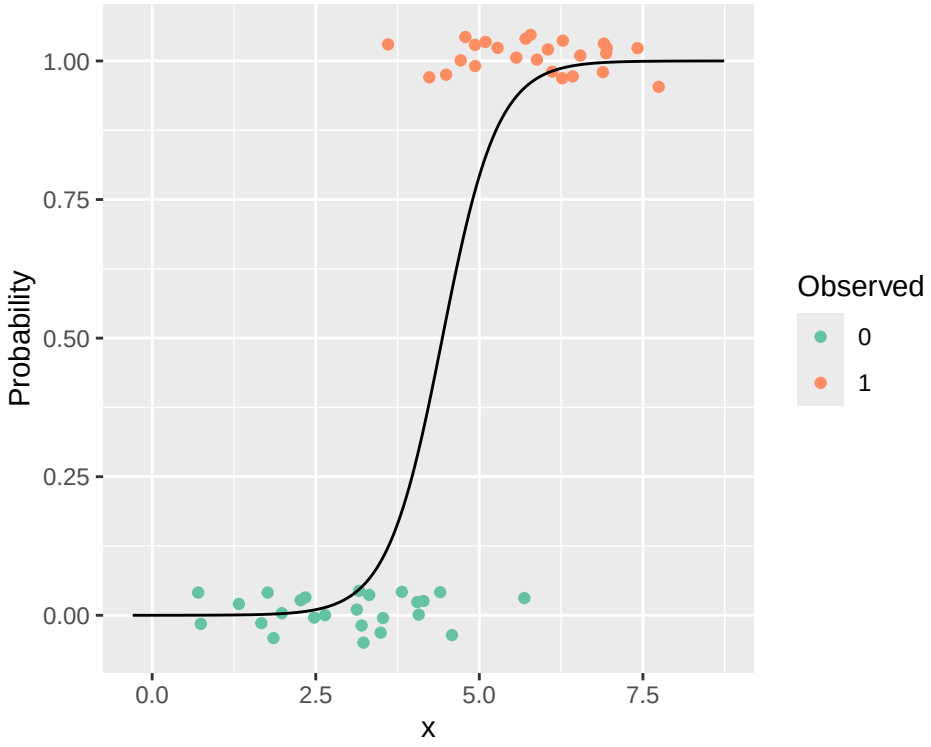
1 Logistic regression

In (normal) linear regression, we predict a continuous numerical variable. Logistic regression extends linear regression to classification, by adapting several things.

1.1 Classification

The aim of classification is to predict the category of an observation based on its predictors. For example, based on the pixels of an image, classify it as a cat or not a cat. Or, based on the contents of an email, classify it as spam or not spam. For this course, we will looking at only two possible outcomes (categories) like the above. Logistic regression for multiple categories is more complicated, and is known as multinomial logistic regression.

For our two-class (binary) classification problems, we will say that our outcome variable y can take on only two values, 1 or 0. In the above example of spam classification, we may say that $y = 1$ corresponds to a message being 'spam', and $y = 0$ corresponds to a message being 'not spam'. We may also say that the interpretations are exactly flipped, i.e. $y = 1$ is a message being 'not spam' and $y = 0$ is a message being 'spam'. So, take great care in understanding what it means when $y = 0$ and $y = 1$ in any model.



1.2 Adapting linear regression to classification

Linear regression supposes that we have a continuous numerical variable y we want to predict with predictors x_i:

$$y = \beta_0 + \beta_1 \times x_1 + \cdots + \beta_p \times x_p$$

In this model, y is unbounded, i.e. can range from $-\infty$ to $+\infty$. This makes the model slightly inappropriate for classification problems, since our y can only be 0 or 1.

Since y can only be 0 or 1, we will consider and aim to model the probability of an observation being 0 or 1, i.e. $P(Y = 0)$ and $P(Y = 1)$. Since there are only two possibilities, $P(Y = 0) + P(Y = 1) = 1 \implies P(Y = 1) = 1 - P(Y = 0)$ Also note that these probabilities are bounded, and must be between 0 and 1, i.e. $0 \leq P(Y = 0), P(Y = 1) \leq 1$.

To adapt linear regression to classification with probabilities, we must somehow map this bounded range of probability (0 to 1) to the unbounded range of linear regression ($-\infty$ to $+\infty$) (and vice versa) This requires two elements, which are covered in the next two sections:

1. the odds (and the log-odds); and
2. the logistic function.

1.2.1 The odds ratio

For our binary classification, the odds ratio represents the ratio of the probability of success (say $Y = 1$) to failure ($Y = 0$).

$$\frac{P(Y = 1)}{P(Y = 0)} = \frac{P(Y = 1)}{1 - P(Y = 1)}$$

The range of the odds ratio is $[0, +\infty)$. If the odds are greater than 1, there is a greater probability of success than failure. If the odds are less than 1, failure is more likely.

If we apply the natural log function (log base e), we get the log-odds, and its range will become $(-\infty, +\infty)$. Since $log(1) = 0$, the interpretation of the above is similar. If the log-odds are greater than 0, there is a greater probability of success than failure. If the log-odds are less than 0, failure is more likely.

The log-odds are an appropriate target for our linear regression, since its range is unbounded. Using linear regression, we can model the log-odds using our predictors x_i. From the modeled log-odds, we can perform a few transformations to then get our probability of success $P(Y = 1)$.

We also call the **log-odds** the **logit**.

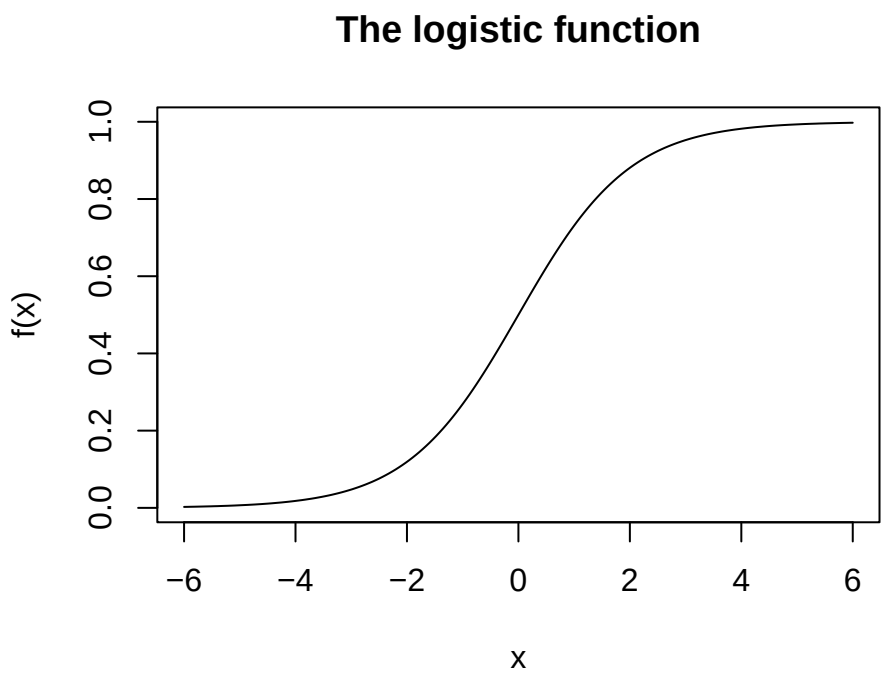
$$\log\left(\frac{P(Y = 1)}{P(Y = 0)}\right) = \log\left(\frac{P(Y = 1)}{1 - P(Y = 1)}\right) = \beta_0 + \beta_1 \times x_1 + \cdots + \beta_p \times x_p$$

1.2.2 The logistic function

The logistic function is an example of a *sigmoidal function*, which is a family of functions with a distinctive S-shaped curve. It takes in a single numerical value, and maps it to some value between 0 and 1.

Applying this logistic function to the log-odds will yield the probability of success $P(Y = 1)$.

$$f : (-\infty, +\infty) \to [0, 1] \quad f(x) = \frac{1}{1 + e^{-x}}$$



2 The `BreastCancer` dataset

For this tutorial, we are looking at the `BreastCancer` dataset in the `mlbench` package, which contains measurements of breast cancer cases, and whether they were malignant or benign. Looking at the structure of the data, we can see there are 699 observations and 11 variables, all of which are *factors* (categorical).

They are stored as factors because *the distance between each may not be uniform*. A simple example: a fan may have 3 settings, 'High', 'Medium' and 'Low'. The fan may not spin faster linearly based on the setting. The same occurs here: the difference between 1 and 2 may be different from between 2 and 3, and between 9 and 10, but there is a *natural order* to them.

Looking at the summary of the data, we can also see that `Bare.nuclei` has 16 missing values. This is also highlighted in the dataset information, which you can see with `?mlbench::BreastCancer`. Whether to remove these entries with missing values is up to you. Even though we are not using this feature in our model, these missing values *could* represent a failure in the data collection process, and mean that those observations are bad data.

```r
data('BreastCancer', package='mlbench')
str(BreastCancer)
```

```
## 'data.frame':    699 obs. of  11 variables:
##  $ Id             : chr  "1000025" "1002945" "1015425" "1016277" ...
##  $ Cl.thickness   : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<..: 5 5 3 6 4 8 1
##  $ Cell.size      : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<..: 1 4 1 8 1 10
##  $ Cell.shape     : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<..: 1 4 1 8 1 10
##  $ Marg.adhesion  : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<..: 1 5 1 1 3 8 1
##  $ Epith.c.size   : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<..: 2 7 2 3 2 7 2
##  $ Bare.nuclei    : Factor w/ 10 levels "1","2","3","4",..: 1 10 2 4 1 10 10
##  $ Bl.cromatin    : Factor w/ 10 levels "1","2","3","4",..: 3 3 3 3 3 9 3 3 1
##  $ Normal.nucleoli: Factor w/ 10 levels "1","2","3","4",..: 1 2 1 7 1 7 1 1 1
##  $ Mitoses        : Factor w/ 9 levels "1","2","3","4",..: 1 1 1 1 1 1 1 1 5
##  $ Class          : Factor w/ 2 levels "benign","malignant": 1 1 1 1 1 2 1 1
```

```r
summary(BreastCancer)
```

```
##       Id             Cl.thickness   Cell.size     Cell.shape  Marg.adhesion
##  Length:699         1      :145   1      :384   1      :353   1      :407
##  Class :character   5      :130   10     : 67   2      : 59   2      : 58
##  Mode  :character   3      :108   3      : 52   10     : 58   3      : 58
##                     4      : 80   2      : 45   3      : 56   10     : 55
##                     10     : 69   4      : 40   4      : 44   4      : 33
##                     2      : 50   5      : 30   5      : 34   8      : 25
##                     (Other):117   (Other): 81   (Other): 95   (Other): 63
##   Epith.c.size  Bare.nuclei   Bl.cromatin  Normal.nucleoli    Mitoses
##  2      :386   1      :402   2      :166   1      :443     1      :579
##  3      : 72   10     :132   3      :165   10     : 61     2      : 35
##  4      : 48   2      : 30   1      :152   3      : 44     3      : 33
##  1      : 47   5      : 30   7      : 73   2      : 36     10     : 14
##  6      : 41   3      : 28   4      : 40   8      : 24     4      : 12
##  5      : 39   (Other): 61   5      : 34   6      : 22     7      :  9
##  (Other): 66   NA's   : 16   (Other): 69   (Other): 69     (Other): 17
##        Class
##  benign   :458
##  malignant:241
##
##
##
##
##
```

3 Preparing the dataset

Following the tutorial, we will remove these observations with missing values. There are multiple ways to do this, indexing the indices returned by `complete.cases()`, or using `na.omit()`. Both of these yield the same dataset.

We will also extract out the features we are interested in and convert them to numerical variables with `sapply()`, which applies a function FUN to each element of its input, in this case, the 2nd to 4th features of `bc`. The outcome variable of interest `Class` is also converted into `1`s or `0`s based on whether they were malignant or benign, with `1` representing malignant cancers. This conversion into 1s and 0s is not strictly necessary - we could have kept the classes as 'malignant' and 'benign'.

```
bc <- na.omit(BreastCancer)
bc <- BreastCancer[complete.cases(BreastCancer),]

# bc[,2] <- as.numeric(bc[,2])
bc[,2:4] <- sapply(bc[,2:4], as.numeric)
bc <- bc %>% mutate(y = factor(ifelse(Class=="malignant", 1, 0))) %>%
  select(Cl.thickness:Cell.shape, y)
str(bc)
```

```
## 'data.frame':    683 obs. of  4 variables:
##  $ Cl.thickness: num  5 5 3 6 4 8 1 2 2 4 ...
##  $ Cell.size   : num  1 4 1 8 1 10 1 1 1 2 ...
##  $ Cell.shape  : num  1 4 1 8 1 10 1 2 1 1 ...
##  $ y           : Factor w/ 2 levels "0","1": 1 1 1 1 1 2 1 1 1 1 ...
```

4 Training the model

Our model for the log-odds on whether a breast cancer case is malignant or benign is:

$$\log\left(\frac{P(\text{malignant})}{P(\text{benign})}\right) = \beta_0 + \beta_1 \times \text{thickness} + \beta_2 \times \text{size} + \beta_3 \times \text{shape}$$

We follow through with the standard process of training a model, starting with a train-test split.

```
set.seed(100)
train.idx <- sample(nrow(bc), 0.8*nrow(bc))
train.data <- bc[train.idx,]
test.data <- bc[-train.idx,]
```

We will use the built-in `glm()` function for our logistic regression. It is similar to the built-in `lm()` function, but for our purposes, accepts an additional `family` parameter. For a logistic regression, we will use `family='binomial'`. The interpretation of the model is similar to linear regression, but

for the **log-odds** instead of the outcome variable directly. To get the change in the odds, exponentiate it by e. Here, only the `Estimate` column and columns with stars $\star$ are important for the course.

```
m.logistic <- glm(y ~ ., data=train.data, family='binomial')
summary(m.logistic)
```

```
##
## Call:
## glm(formula = y ~ ., family = "binomial", data = train.data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -7.7184     0.7699 -10.025  < 2e-16 ***
## Cl.thickness   0.5195     0.1121   4.634 3.59e-06 ***
## Cell.size      0.7081     0.2106   3.362 0.000773 ***
## Cell.shape     0.7679     0.1971   3.895 9.80e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 703.10  on 545  degrees of freedom
## Residual deviance: 138.33  on 542  degrees of freedom
## AIC: 146.33
##
## Number of Fisher Scoring iterations: 7
```

The final model, which can be read from the summary, is below. For each unit increase in `thickness`, the **log-odds** of a tumour being malignant increases by 0.5195, or the **odds** increase by $e^{0.5195} \approx 1.681$ times.

$$\log\left(\frac{P(\text{malignant})}{P(\text{benign})}\right) = -7.7184 + 0.5195 \times \text{thickness} + 0.7081 \times \text{size} + 0.7679 \times \text{shape}$$

5 Evaluating the model

As usual, we will use `predict()` to get the results of our model. Our model is predicting the **log-odds**, so to get the probability of interest (that a cancer case is malignant), we have to specify an extra parameter `type='response'`. If `type='response'` is not included, the log-odds are returned, which you can transform back to probabilities manually.

```
logodds <- predict(m.logistic, test.data)
p <- 1/(1+exp(-logodds))          # transform back to probabilities
head(p)
```

```
##           5          10          17          20          30          34
## 0.015303063 0.030585679 0.015303063 0.042078567 0.014963619 0.005468135
```

```
yhat.p <- predict(m.logistic, test.data, type='response')
head(yhat.p)
```

```
##           5          10          17          20          30          34
## 0.015303063 0.030585679 0.015303063 0.042078567 0.014963619 0.005468135
```

Our `yhat.p` now consists of the probabilities ($P(Y = 1)$) that a given observation (in our test set) is malignant. We must now set some **decision threshold** to determine whether our model predicts a cancer case is malignant or benign. For this tutorial, we will simply choose $p = 0.5$ as our decision threshold. If `yhat > 0.5`, this indicates that it is more likely that an observation is malignant than benign, and so we will classify it as such.

```
yhat <- factor(ifelse(yhat.p > 0.5, 1, 0))
yhat
```

```
##   5  10  17  20  30  34  45  51  53  54  62  64  65  71  72  77  91 101 107 1
##   0   0   0   0   0   0   1   1   1   1   0   1   0   0   1   0   0   1   1
## 128 142 144 147 152 153 155 156 162 164 170 182 184 186 195 201 202 204 205 2
##   0   0   0   1   1   1   0   1   0   0   0   0   1   0   0   1   1   0   0
## 209 213 217 221 222 223 232 238 243 254 260 265 268 274 275 284 288 291 297 3
##   0   0   0   0   1   0   1   1   0   1   1   1   0   1   0   1   0   0   1
## 306 325 327 329 331 333 353 358 362 364 371 376 379 381 382 383 393 399 402 4
##   1   0   0   1   1   0   1   1   1   1   0   0   0   0   1   0   0   0   0
## 424 428 430 449 451 455 469 470 474 478 479 480 483 494 497 502 511 520 523 5
##   0   1   0   0   0   0   0   0   0   0   0   1   1   1   0   0   0   1   1
## 539 541 542 544 546 550 555 559 560 572 576 578 579 581 583 584 586 588 589 5
##   0   0   0   0   0   1   0   0   0   1   0   0   0   0   1   0   0   0   1
## 591 596 605 606 611 619 637 641 648 671 677 678 685 687 693 698 699
##   1   0   0   1   1   0   1   0   0   1   0   0   0   0   0   1   1
## Levels: 0 1
```

5.1 Evaluation metrics

In regression problems, we utilised the RMSE as a metric for our model's performance on the test data. In classification, we can't really compute the RMSE. Instead, we will be looking at the **accuracy** and **confusion matrix**.

5.2 Accuracy

Accuracy is a very natural metric to use - what percentage of cases did the model predict correctly, i.e. true benign cases as benign and true malignant cases as malignant? We simply compare the predicted classes to the true classes, yielding a vector of TRUE and FALSE values. Taking the mean of this vector returns us the number of TRUE (129 such values) over the total number of cases (137 in test data).

```
y <- test.data$y
mean(yhat == y)
```

```
## [1] 0.9416058
```

5.3 Confusion matrix

The confusion matrix is a detailed look at the model's performance, showing us four measures: the *true positive* and *true negative* cases, which the model predicted accurately, and the *false positive* and *false negative* cases, which the model predicted wrongly.

True positive and true negative cases are where the model predicted the outcome class correctly. In this model, false positives are where the model predicted a tumour to be *malignant* when in reality it was *benign*. False negatives here indicate where the model predicted a tumour to be *benign* when in reality it was *malignant*. Both of these, obviously, have much different real-world implications.

To see the confusion matrix, we use `table()`. The first parameter `yhat` is shown along the rows. The second parameter `y` is shown along the columns. For example, the 1st row, 2nd column indicates the **false negatives** (`yhat = 0` when in reality `y = 1`). From this table, we can also compute the accuracy.

```
table(predicted=yhat, observed=y)
```

```
##           observed
## predicted  0  1
##         0 82  4
##         1  4 47
```

5.4 False positive and false negative rates

From this confusion matrix, we can also calculate the false positive and false negative rates. The false positive and false negative rates give us an indication of how the model fails.

These rates are defined with respect to the **true number of observations of each category**. For the false positive rate (FPR), we consider the true number of *negative* observations, since false positives

are in actuality a negative observation. Likewise, for the false negative rate (FNR), we consider the true number of *positive* observations, since false negatives are in actuality a positive observation.

$$\text{FPR} = \frac{\text{FP}}{\text{TN} + \text{FP}} = \frac{\text{FP}}{\text{number of observed positive classes}} \text{FNR} = \frac{\text{FN}}{\text{TP} + \text{FN}} = \frac{\text{FN}}{\text{number of observed negative clas}}$$

Here, we have a false positive rate of $4/86 = 4.76\%$, indicating that we inaccurately tell 4.76% of patients with benign tumours that they have malignant tumours, leading to wasted treatment/investigations.

We have a false negative rate of $4/51 = 7.84\%$, indicating that we inaccurate tell 7.84% of patients with malignant tumours that they have benign tumours. In this case, we fail to address their disease.

5.5 Accuracy, revisited

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TN} + \text{TN} + \text{FP} + \text{FN}} = \frac{\text{TP} + \text{TN}}{\text{number of observations}}$$

The definition for accuracy is extremely natural, just the number of correct cases. However, there are some weaknesses to it.

Consider if our dataset only had 1% of patients having malignant tumours (the positive class). An extremely simple model that predicts everyone having benign tumours (the negative class) would have an accuracy of 99%, while missing every malignant tumour. In this scenario, using accuracy as the only metric may mislead us into thinking our model is actually useful.

Because of this, we consider other evaluation metrics alongside accuracy to get a better understanding where our model performs well, and where it fails.

5.6 Recall/sensitivity

Another evaluation metric we can use is **recall**, also known as sensitivity, which is defined as:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{number of observed true classes}} = 1 - \text{FNR}$$

In the context of this dataset, we have $47/51 = 92.2\%$, indicating we correctly identify 92.2% of malignant tumours for further treatment.

6 Extra: choice of decision threshold

While we fix $p = 0.5$ as our decision threshold, this value is actually another parameter that can be chosen. The choice of this decision threshold has a meaningful real-world interpretation.

In this dataset, for example, we may choose a *lower* p to correspond with being more cautious on whether a breast cancer case is malignant. A lower p value here would result in **more false positives**

but **less false negatives**, i.e. more *actually benign* cases which are predicted to be malignant. You can think of it as being more cautious with people's health. You may apply the same logic of setting a lower decision threshold in predicting whether someone is COVID/tuberculosis/ebola/other infectious disease positive, out of concern for public health.

Setting a *higher* p corresponds to the model having *high confidence* in its predictions. Again with this same dataset, we may say that we want only those that the model strongly believe are malignant cases, leading to **less false positives** but **more false negatives**. False positives (misdiagnosing benign tumors as malignant) lead to unnecessary invasive procedures, patient anxiety, and added healthcare costs. A high threshold minimizes these risks. False negatives (missing malignant cases) are increased with this model, but this is often acceptable when confirmatory tests (like biopsies) are costly or risky.

The choice of this parameter also affects our metrics, including the accuracy. Since this is a parameter of the model, choosing this parameter's value is done in the training process, without use of the test data.